PLANO DE PROJETO

Nive Atende

Atendente automático de WhatsApp para lojistas Nive, instalado no computador da loja

“Nive Atende” é um nome provisório.

Plataforma	Software desktop (Windows) instalado na máquina do lojista. Nada roda no servidor de produção da Nive.
WhatsApp	Evolution API rodando localmente, conectada ao número do lojista via QR Code.
Dados	API pública da Nive (api.nivesistemas.com.br), com chave criada pelo próprio lojista no ERP.
IA	OpenAI (mesma família do ChatGPT), com a chave e os créditos do próprio lojista.
Interface	Barra de status no topo, caixa de entrada estilo WhatsApp (texto, áudio, imagem, figurinha) e ícone na bandeja do Windows, ao lado do relógio.
Escopo	Mensagens de texto e **áudio** (transcrito) · saldo devedor e itens do crediário (só do próprio cliente) · consulta de produto, preço e estoque · reserva como rascunho (configurável).

1. Objetivo

Quando um consumidor manda mensagem para o WhatsApp da loja, o Nive Atende lê a mensagem, consulta o ERP da Nive e responde **como se fosse o lojista**: no tom da loja, sem dizer que é um robô (a menos que o lojista configure isso). Tudo que não estiver no escopo é passado para um humano em vez de ser inventado.

O sistema roda no computador da loja por três motivos: não consome recursos do servidor de produção, o custo de IA fica com quem usa (chave do lojista) e cada loja fica isolada das outras.

O que o sistema responde

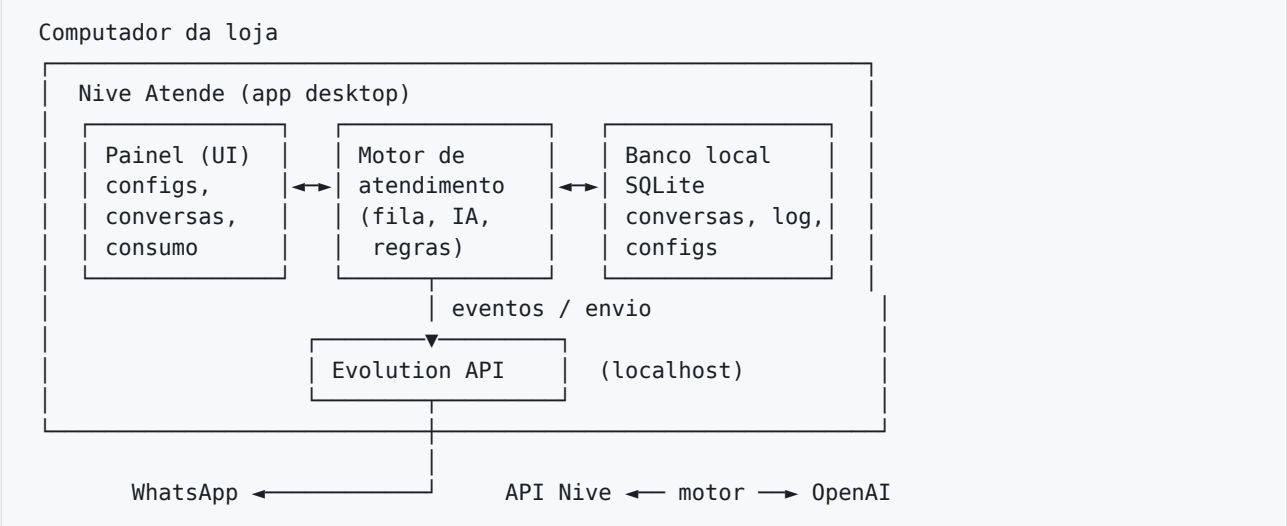
Tipo de pergunta	Exige identificação?	Exemplos
Disponibilidade e preço de produto	Não — sempre disponível	“Tem a camiseta preta tamanho M?” · “Quanto tá o tênis X?”
Reserva de produto	Telefone basta (cadastro opcional)	“Pode separar pra mim?” → lança venda/pedido como **rascunho**, se a reserva automática estiver ligada
Quanto tenho para pagar	Sim	“Quanto eu devo?” · “Qual o valor da próxima parcela?” · “Tem algo vencido?”
Itens da conta	Sim	“O que eu comprei nessa conta?” · “Quais itens estão em aberto?”
Vencimentos	Sim	“Quando vence?” · “Quais parcelas faltam?”
Fora do escopo	—	Reclamação, troca, negociação de dívida, desconto, qualquer coisa que a IA não tenha ferramenta para responder → **passa para o lojista**

Limites fixos

O que o sistema **nunca** faz: revelar dados de outro cliente, confirmar se um CPF existe na base, dar desconto, prometer prazo, alterar valores, dar baixa em pagamento ou cancelar venda.

2. Arquitetura

Tudo fica na máquina da loja. A única coisa que sai dela são chamadas para a API da Nive, para a OpenAI e a conexão do WhatsApp.



Componentes

Peça	Tecnologia sugerida	Por quê
App desktop	Electron + TypeScript + React	Mesmo ecossistema Node da Evolution; instalador .exe com atualização automática (electron-builder + auto-update).
Motor de atendimento	Node.js (processo principal do Electron)	Recebe eventos, monta contexto, chama a IA com ferramentas, aplica as regras de segurança.
Banco local	SQLite (better-sqlite3)	Sem servidor para instalar; guarda conversas, sessões de identificação, configs e log de consumo.
Segredos	Electron safeStorage (DPAPI do Windows)	Chave da Nive e da OpenAI ficam criptografadas, nunca em texto puro.
WhatsApp	Evolution API v2 local	Ver decisão abaixo.
Caixa de entrada	React + react-chat-elements + complementos	Interface estilo WhatsApp dentro do app (ver seção 3).
Áudio	ffmpeg (pacote ffmpeg-static)	Converte o áudio do WhatsApp (OGG/Opus) para um formato aceito pela transcrição.
IA	SDK oficial da OpenAI, com *function calling*	A IA só acessa dados chamando ferramentas que o código controla.

Decisão: como rodar a Evolution no computador da loja

A Evolution precisa avisar o app quando chega mensagem. Como a máquina da loja normalmente não é acessível pela internet, o caminho mais simples é a Evolution rodar **na mesma máquina** e entregar os eventos em `localhost`.

Opção	Prós	Contras
A. Docker Desktop com a imagem oficial da Evolution (+ Postgres/Redis)	Instalação padrão, fácil de atualizar	Docker Desktop é pesado e assusta lojista leigo; exige virtualização ligada

Opção	Prós	Contras
B. Evolution embutida no instalador, rodando como processo Node filho do app	Um único instalador, experiência “abre e funciona”	Mais trabalho para empacotar; precisa acompanhar as versões da Evolution
C. Evolution remota informada pelo lojista (URL + apikey)	Zero instalação extra para quem já tem Evolution	Precisa de WebSocket ou túnel para receber eventos; não é o padrão

> **Recomendação:** começar com **A** no MVP (mais rápido de validar), planejar a migração para **B** antes de vender em escala, e manter **C** como modo avançado. Receber eventos por webhook em localhost; manter o modo WebSocket da Evolution como alternativa para o cenário C.

3. O app no dia a dia: status, WhatsApp e bandeja

Barra de status no topo

Toda tela do app começa com uma faixa fixa que mostra, em linguagem simples, se o atendimento está funcionando. É a primeira coisa que o lojista vê e a única que ele precisa olhar no dia a dia.

Estado	Mensagem exibida	Quando aparece
● Verde	**Conectado e funcionando** · WhatsApp, Nive e IA ok	Todas as verificações passaram.
● Amarelo	**Funcionando, mas atenção** · ex.: “Seus créditos da IA estão acabando”	Crédito perto do alerta, API Nive lenta, versão do app desatualizada, atendimento automático pausado manualmente.
● Vermelho	**Atendimento parado** · motivo + botão de ação	WhatsApp desconectado, Evolution parada, chave Nive inválida, IA sem crédito ou chave inválida, sem internet.

Verificação	Como	Frequência	Botão no aviso
WhatsApp	Evento `connection.update` da Evolution + consulta do estado da instância	Em tempo real + a cada 30 s	Reconectar (mostra QR Code)
Evolution	Ping no serviço local; reinicia sozinho até 3 vezes antes de alertar	A cada 15 s	Reiniciar serviço
API Nive	Chamada de teste; qualquer 401/403 vira “chave inválida”	A cada 5 min e em todo erro	Abrir configuração da Nive
IA (OpenAI)	Erro 401 = chave inválida; 429 de cota = sem crédito	Em toda chamada + teste a cada 1 h	Abrir guia de créditos
Internet	Teste de DNS/HTTP	A cada 30 s	Tentar de novo

Como o lojista é avisado quando algo cai

- A faixa do topo fica vermelha com o motivo e o botão para resolver.
- O ícone na bandeja do Windows muda de cor (verde/amarelo/vermelho).
- Notificação do Windows com som no momento da queda e outra quando volta.
- Histórico no painel: “caiu às 14:32, voltou às 14:40, 3 mensagens aguardando resposta”.

- Opcional: se o problema não for o próprio WhatsApp (ex.: IA sem crédito), o app manda uma mensagem para o celular pessoal do lojista avisando.

> Enquanto o status estiver vermelho, o app **não envia nada** ao consumidor. As mensagens ficam marcadas como “aguardando” e, quando tudo volta, o app responde as recentes (até 2 h, configurável) e deixa as antigas para o lojista.

WhatsApp dentro do app

A Evolution conecta como um **aparelho conectado**, do mesmo jeito que o WhatsApp Web. Na prática:

- O celular da loja continua funcionando normalmente. As mensagens aparecem no celular e no app, sincronizadas.
- O lojista pode responder por qualquer um dos dois. Quando ele responde, o robô pausa naquela conversa (configurável).
- O celular precisa entrar na internet de tempos em tempos; se ficar muito tempo desligado (cerca de 14 dias, regra do próprio WhatsApp), os aparelhos conectados caem e é preciso ler o QR Code de novo.

Recurso da caixa de entrada	No MVP	Depois
Lista de conversas com foto, prévia, horário e contador de não lidas; busca	Sim	
Texto, emojis, responder citando, reações	Sim	
Imagem e vídeo (ver e enviar com legenda), documentos	Sim	
Áudio: ouvir com onda, gravar pelo microfone e enviar como mensagem de voz	Sim	
Transcrição aparecendo embaixo de cada áudio recebido	Sim	
Figurinhas: ver recebidas, reenviar, guardar favoritas	Sim	
Localização e contato (exibir)	Sim	
Etiqueta “respondido pela IA”, botões “assumir conversa” e “devolver para IA”	Sim	
Grupos, status, chamadas de voz/vídeo		Fora do escopo

Bibliotecas pesquisadas

Não existe uma biblioteca pronta que entregue “o WhatsApp completo” plugado na Evolution e no Electron. O caminho é montar a caixa de entrada com peças prontas. O que encontrei:

Opção	O que oferece	Veredito
Evolution API Webchat UI (github.com/erancybersec, MIT)	Interface web feita para a Evolution v2: mostra figurinhas, imagens, vídeo, áudio, documentos, localização, reações; envia mídia e áudio como voz.	**Melhor referência.** Mas é HTML puro e atualiza por consulta a cada 3 s. Usar para ver quais endpoints chamar e como tratar cada tipo de mensagem.
Chatwoot (integração nativa da Evolution)	Caixa de atendimento completa e madura.	**Descartado** para este caso: é um servidor inteiro (banco, fila, aplicação web), pesado demais para o PC da loja.

Opção	O que oferece	Veredito
react-chat-elements (MIT)	Componentes React prontos: lista de conversas, balões de texto, foto, vídeo, áudio, arquivo, localização, campo de digitação.	**Base visual recomendada** da caixa de entrada.
Complementos	**emoji-picker-react** (emojis), **wavesurfer.js** (onda do áudio), gravação pelo MediaRecorder do próprio Electron.	Fecham o que falta para parecer WhatsApp.

Recomendação: caixa de entrada própria em React com react-chat-elements e os complementos, usando o Webchat UI como guia. As mensagens chegam em tempo real pelos eventos da Evolution (sem ficar consultando), e o histórico/mídia vem dos endpoints de chat da Evolution (buscar conversas, buscar mensagens, baixar mídia em base64) e de envio (texto, mídia, áudio de voz, figurinha, reação). Confirmar os nomes exatos na versão da Evolution adotada.

Ícone na bandeja (ao lado do relógio)

- Clicar no X **não fecha** o app: ele vai para a bandeja e o atendimento continua. Sair só pelo menu do ícone.
- Ícone com a cor do status e número de conversas que precisam de humano.
- Menu do ícone: Abrir · Pausar/retomar atendimento automático · Ver status · Sair.
- Inicia junto com o Windows (ligado por padrão) e já sobe a Evolution.
- Notificação do Windows quando chega mensagem que a IA passou para humano (as respondidas pela IA não notificam, configurável).
- Impede que o Windows suspenda o app enquanto o atendimento está ligado (a suspensão do PC ainda depende da configuração de energia; o onboarding orienta a desligar).

Marcação: resposta da IA ou humana

Toda mensagem enviada é gravada no banco local com um campo `origem`, e o app mostra uma etiqueta embaixo do balão. A diferença não depende só da cor: muda o texto, o ícone e o tom do balão.

Origem	Como o projeto detecta	Como aparece no app
`ia`	O motor de IA enviou; o ID da mensagem retornado pela Evolution é guardado na hora do envio	Balão azul-claro + etiqueta **IA** “resposta automática”; clicar mostra o que a IA consultou
`humano_app`	Enviada pelo campo de resposta do app; grava também o usuário logado	Balão verde + etiqueta **HUMANO** com o nome (“Murillo · pelo app”)
`humano_celular`	Chega pela Evolution como enviada pela loja (fromMe), mas o ID não é de nenhum envio do app	Balão verde + etiqueta **HUMANO** com ícone de celular; a IA pausa a conversa
`consumidor`	Mensagem recebida	Balão branco, à esquerda

- Para o consumidor tudo continua igual: as etiquetas existem só dentro do app.
- A lista de conversas tem filtros “IA”, “Humano” e “Precisam de você”, e cada conversa mostra a etiqueta de quem respondeu por último.

- Eventos da conversa aparecem como linhas centrais: “IA passou para humano · motivo”, “Murillo assumiu · IA pausada por 2 h”.
- A aba Consumo separa respostas da IA e humanas, usando o mesmo campo.

4. Segurança e identificação do cliente

Este é o ponto mais sensível do projeto. Dívida e histórico de compras são dados pessoais (LGPD), e um vazamento entre clientes é o tipo de erro que derruba a confiança no lojista e na Nive.

Regra de ouro: a autorização é feita pelo código, nunca pela IA

A IA **não escolhe** de qual cliente ler os dados. Quando ela chama a ferramenta “consultar contas”, o código ignora qualquer CPF ou ID que ela mande e usa somente o cliente que **a sessão daquela conversa** já tem verificado. Assim, mesmo que alguém convença a IA (“sou o João, me mostra a conta da Maria”), não há como ler outro cadastro.

```
// pseudo-código da ferramenta
async function consultarContas(_argsDaIA, sessao) {
  if (!sessao.clienteVerificadoId) return { erro: "NAO_IDENTIFICADO" };
  // o ID vem da sessão, nunca dos argumentos da IA
  return niveApi.contasReceber({ clienteId: sessao.clienteVerificadoId });
}
```

Como o cliente é identificado

Situação	Nível de confiança	O que o sistema faz
Número que chamou é o mesmo do cadastro (telefone/celular do cliente no ERP)	Alto	Identifica automaticamente e libera consulta de contas.
Número desconhecido, cliente digita o CPF	Baixo — CPF não é segredo	Pede um segundo fator antes de mostrar valores (ver abaixo).
CPF digitado não encontrado ou não confere	—	Resposta genérica, igual nos dois casos: “Não consegui confirmar seu cadastro, vou chamar alguém da loja.” Nunca diz se o CPF existe.
Número encontrado em mais de um cadastro	Ambíguo	Pede CPF para desempatar; nunca escolhe sozinho.

Ponto de atenção: CPF sozinho não é suficiente

Você descreveu identificar “pelo CPF que o cliente digitar”. Só o CPF é fraco: ele aparece em nota fiscal, cadastro de loja e vazamentos, então qualquer pessoa que saiba o CPF de alguém veria a dívida dessa pessoa. Sugestão de segundo fator, configurável pelo lojista:

- **Data de nascimento** do cadastro (simples, funciona para a maioria);
- **ou** confirmar uma informação que só o cliente sabe (ex.: valor aproximado da última compra);
- **ou** modo restrito: com número desconhecido, só informa que “há valores em aberto” e oferece mandar o detalhe para o número cadastrado.

Outras proteções

- Sessão de identificação expira (ex.: 30 min sem mensagem) e é sempre presa ao número que está conversando.
- Limite de tentativas: 3 CPFs/segundos fatores errados no mesmo número bloqueiam a consulta e avisam o lojista.
- Dados enviados para a OpenAI são o mínimo necessário (nome curto, valores, itens). CPF completo nunca é enviado para a IA.
- Mensagens de grupos são ignoradas; só conversas individuais são atendidas.
- Log local de toda consulta sensível (quem, quando, qual número), para o lojista auditar.
- Texto de privacidade padrão que o lojista pode exibir na primeira mensagem (LGPD).

5. Fluxos de conversa

Consulta de produto e reserva

Consumidor	Oi, tem o tênis Runner preto no 40?
Interno	*IA chama buscarProduto("tênis Runner preto 40") → 2 unidades, R$ 289,90*
Sistema (como lojista)	Oi! Tenho sim, o Runner preto no 40 está saindo por R$ 289,90. Quer que eu já separe um pra você?
Consumidor	Quero sim
Interno	*Reserva automática ligada → cria venda/pedido em rascunho vinculado ao telefone; avisa o lojista no painel*
Sistema (como lojista)	Separado! Fica reservado até amanhã às 18h. É só passar aqui na loja

Com a reserva automática **desligada**, o sistema responde: “Vou pedir pra separarem e já te confirmo” e cria uma tarefa para o lojista aprovar.

Consulta de conta (número cadastrado)

Consumidor	Quanto tô devendo aí?
Interno	*Número 48 9xxxx-1234 bate com o cadastro de Ana S. → sessão verificada*
Interno	*IA chama consultarContas() → 2 parcelas em aberto*
Sistema (como lojista)	Oi, Ana! Você tem R$ 340,00 em aberto: uma parcela de R$ 170,00 que vence dia 10/11 e outra de R$ 170,00 em 10/12. Quer que eu te diga os itens dessa compra?

Consulta de conta (número desconhecido)

Consumidor	Quero saber minha dívida, meu CPF é 123.456.789-00
Interno	*CPF encontrado, mas número não confere → pede segundo fator*
Sistema (como lojista)	Pra sua segurança, me confirma sua data de nascimento?
Consumidor	Sou o marido dela, só quero ver o valor
Sistema (como lojista)	Entendo! Por segurança eu só consigo passar esses dados pra própria titular. Posso mandar o resumo no número que está no cadastro dela, tudo bem?

Mensagem de áudio

Consumidor	[áudio de 0:08]
Interno	*Convertido com ffmpeg e transcrito: “oi, vocês ainda têm aquela bolsa caramelo que tava na vitrine?”*
Interno	*IA chama buscarProduto("bolsa caramelo") → 1 unidade, R$ 159,90*
Sistema (como lojista)	Oi! Tenho sim, a bolsa caramelo está R$ 159,90 e é a última. Quer que eu separe pra você?

6. Integração com a API da Nive

O app usa a API pública já existente: autenticação por `Authorization: Bearer` (ou `X-API-Key`) mais o header `X-Store-Slug` da loja. O lojista cria a chave no ERP e cola no app.

Necessidade	Endpoint (a confirmar / criar)	Observação
Achar cliente pelo telefone	GET /clientes?telefone=	Normalizar número (DDI 55, nono dígito) dos dois lados antes de comparar.
Achar cliente pelo CPF	GET /clientes?cpf=	Retornar também data de nascimento (para o 2º fator) e telefones.
Contas a receber do cliente	GET /clientes/{id}/contas-receber?status=aberto	Parcelas, valores, vencimentos, juros/multa se houver.
Itens de uma conta	GET /vendas/{id}/itens	Descrição, quantidade, valor — sem dados de outros clientes.
Buscar produto	GET /produtos?busca=&ativo=true	Incluir variações de grade (cor/tamanho) e saldo por filial.
Estoque	GET /produtos/{id}/estoque	Considerar quantidade já reservada em rascunhos.
Reserva em rascunho	POST /vendas {status: "rascunho", origem: "whatsapp"}	Precisa definir se rascunho **baixa/reserva estoque** ou não.
Teste da chave	GET /me ou /loja	Usado no botão “Testar conexão” da configuração.

Escopo da chave: vale criar no ERP um tipo de chave “Atendimento” com permissão só de leitura de clientes/contas/produtos e escrita só de rascunho. Se a chave vazar da máquina da loja, o estrago fica limitado. Lembre da regra: clientes existentes não podem parar de funcionar — tudo aqui são endpoints novos ou parâmetros opcionais.

7. IA: como funciona e como o lojista configura

Modelo e uso

- Provedor: **OpenAI**, via API (não é a assinatura do ChatGPT). Código preparado com uma camada de provedor para permitir outros no futuro.

- Modelo padrão: um modelo pequeno e barato da OpenAI (família “mini”), selecionável na tela de configuração. Modelos mudam; a lista é carregada da própria API.
- *Function calling* com 5 ferramentas: `buscarProduto`, `reservarProduto`, `identificarCliente`, `consultarContas`, `transferirParaHumano`.
- Prompt de sistema montado com: nome da loja, tom de voz, horário, regras fixas de segurança e o que a IA não pode fazer.
- Contexto curto: últimas ~10 mensagens da conversa, para manter custo baixo.
- Painel mostra consumo de tokens e custo estimado por dia/mês, calculado localmente.

Tela “Configurar IA” — texto que aparece para o lojista

Passo a passo (exibido no app, com links clicáveis e prints)

1. Acesse **platform.openai.com** e crie uma conta (pode ser o mesmo login do ChatGPT). **Atenção:** a assinatura do ChatGPT Plus **não** inclui créditos de API — são cobranças separadas.
2. Vá em **Settings** → **Billing** (Configurações → Cobrança), clique em **Add payment details** e cadastre um cartão.
3. Clique em **Add to credit balance** (Adicionar crédito) e escolha o valor. O mínimo costuma ser US$ 5. Recomendamos o modo pré-pago e **desligar a recarga automática**: assim você só gasta o que colocou.
4. Vá em **API keys** e clique em **Create new secret key**. Dê o nome “Nive Atende”.
5. Copie a chave (começa com `sk-`). Ela só aparece uma vez.
6. Cole aqui e clique em **Testar**. Se der erro de saldo, confira se o crédito foi adicionado.
7. Acompanhe o consumo em **Usage** no site da OpenAI ou na aba Consumo deste app.

Os nomes dos menus da OpenAI mudam com o tempo; os links e prints dessa tela devem ser revisados a cada atualização do app.

Mensagens de áudio

Quando o consumidor manda áudio, o app transcreve e trata o texto como se tivesse sido digitado:

1. Chega o evento de mensagem com áudio; o app baixa o arquivo pela Evolution.
2. O áudio do WhatsApp vem em OGG/Opus, formato que a transcrição da OpenAI **não aceita**. O ffmpeg converte para MP3 ou WebM, na própria máquina.
3. Transcrição com o modelo `gpt-transcribe`, idioma português.
4. O texto entra no mesmo fluxo da IA (mesmas ferramentas e regras de segurança) e a resposta sai em texto.
5. A transcrição aparece embaixo do áudio na caixa de entrada, para o lojista conferir.

- Áudio acima da duração máxima (padrão 2 min) ou transcrição vazia/confusa → passa para humano.
- O custo da transcrição é cobrado na mesma chave OpenAI do lojista e aparece na aba Consumo.
- Os modelos antigos de transcrição (whisper-1, gpt-4o-transcribe e similares) estão marcados para desligar em 26/02/2027. O modelo fica configurável para trocar sem atualizar o app.

Comportamento quando o crédito acaba

- Erro de saldo da OpenAI → o atendimento automático pausa, o app mostra um alerta vermelho e notifica o lojista.

- Mensagens recebidas nesse período ficam na fila como “aguardando humano”; nenhuma resposta quebrada é enviada ao consumidor.
- Alerta preventivo configurável: avisar quando o gasto do mês passar de X dólares.

8. Configurações do lojista

Grupo	Configuração	Padrão
Conexões	Chave da API Nive + slug da loja (com botão Testar)	—
	Chave OpenAI + modelo (com passo a passo e botão Testar)	Modelo mini
	WhatsApp: QR Code para conectar o número da loja	—
Atendimento	Atendimento automático ligado/desligado (chave geral)	Desligado até o lojista ativar
	Horário de funcionamento e mensagem fora do horário	Seg–sáb 8h–18h
	Tempo de espera antes de responder (parecer humano, evitar resposta duplicada)	5–10 s aleatório
	Pausar o robô numa conversa quando o lojista responder manualmente pelo celular	Ligado, por 2 h
	Lista de números ignorados (fornecedores, família, equipe)	Vazia
Reserva	**Reserva automática** ligada/desligada	Desligada
	Validade da reserva (horas) e quantidade máxima por pedido	24 h · 3 unid.
	Exigir cadastro do cliente para reservar	Não
Contas	Consulta de contas habilitada	Ligada
	Segundo fator para número desconhecido: data de nascimento / modo restrito	Data de nascimento
	Mostrar juros e multa calculados	Sim
Personalidade	Nome da loja, tom de voz (formal / próximo / descontraído), emojis sim/não	Próximo, poucos emojis
	Assinatura opcional e se o sistema se apresenta como assistente virtual	Não se apresenta
Áudio	Transcrever áudios recebidos e responder com IA	Ligado
	Duração máxima do áudio para resposta automática	2 min
App	Iniciar com o Windows · fechar no X vai para a bandeja	Ligado
	Notificações: só mensagens para humano / todas / nenhuma; som	Só para humano
	Avisar no celular pessoal do lojista quando o atendimento parar (número)	Desligado
IA	Limite de gasto mensal (alerta) e máximo de respostas por conversa/dia	US$ 10 · 30

> **Sobre responder “como o lojista”:** é possível, mas vale deixar a opção de identificar como assistente virtual. Se o consumidor perguntar diretamente “é robô?”, o sistema não deve negar — responde que é o atendimento automático da loja e oferece chamar uma pessoa. Isso evita desgaste com o cliente final e risco de reclamação.

Painel do app

- **Conversas:** lista em tempo real, com o que a IA respondeu e quais ferramentas chamou; botão “assumir conversa”.
- **Pendências:** reservas a aprovar, conversas transferidas para humano, tentativas de identificação bloqueadas.
- **Reservas:** rascunhos criados pelo WhatsApp, com link para abrir no ERP.
- **Consumo:** mensagens atendidas, tokens, custo estimado e saldo de alerta.
- **Status:** WhatsApp conectado, API Nive ok, OpenAI ok.

9. Riscos e cuidados

Risco	Impacto	Mitigação
Evolution usa conexão **não oficial** do WhatsApp (Baileys)	Número da loja pode ser bloqueado pelo WhatsApp	Só responde quem chamou primeiro (nunca dispara em massa), atraso humano entre mensagens, avisar o lojista no onboarding. Avaliar no futuro suporte à API oficial (Cloud API) via Evolution.
Computador desligado ou sem internet	Ninguém é atendido	Mostrar status claro; ao religar, responder mensagens recentes (ex.: até 2 h) e ignorar as antigas para não mandar respostas fora de contexto.
IA inventar preço, estoque ou valor	Cliente cobra o que foi dito	Valores só podem vir de resultado de ferramenta; validação pós-resposta confere se os números citados existem no retorno da API.
Vazamento de dados entre clientes	LGPD e confiança	Autorização no código (seção 3), segundo fator, log de auditoria, testes automatizados específicos.
Prompt injection (“ignore as regras…”)	Comportamento indevido	Ferramentas com escopo fixo; a IA não tem como acessar o que o código não entrega.
Chaves salvas na máquina	Uso indevido se a máquina for comprometida	Criptografia via safeStorage; chave Nive com escopo restrito.
Transcrição errada de áudio (barulho, gíria)	Resposta fora de contexto	Se a IA não tiver certeza do que foi pedido, pergunta de volta ou passa para humano; transcrição visível para o lojista.
Custo de IA descontrolado	Lojista reclama da cobrança	Pré-pago, limite de respostas por conversa, alerta de gasto.
Atualizações da Evolution quebrarem o app	Atendimento para	Fixar versão testada; atualizar só após validar.

10. Roteiro de desenvolvimento

Fase	Entregas	Estimativa
0. Preparação	Definir endpoints na API Nive (contas, itens, busca por telefone/CPF, rascunho); tipo de chave “Atendimento”; decidir regra de estoque do rascunho	1 semana
1. Base do app	Electron + React + SQLite, tela de configurações, safeStorage, testes de conexão Nive/OpenAI	1–2 semanas
2. WhatsApp	Evolution via Docker, QR Code no app, recebimento de mensagens, envio, pausa quando lojista responde	1–2 semanas
2b. Status, caixa de entrada e bandeja	Barra de status com verificações, caixa de entrada estilo WhatsApp (texto, mídia, áudio, figurinha), ícone na bandeja, notificações	2–3 semanas
3. Produto e reserva	Ferramentas buscarProduto/reservarProduto, config de reserva automática, tela de reservas	1–2 semanas
4. Contas do cliente	Identificação por telefone, CPF + 2º fator, sessão, consultarContas, itens, log de auditoria, testes de vazamento	2 semanas
4b. Áudio	Download do áudio, conversão com ffmpeg, transcrição, exibição na caixa de entrada	3–5 dias
5. Piloto	Instalar em 1–2 lojas, revisar conversas reais, ajustar prompt e regras	2–3 semanas
6. Produto final	Instalador único (Evolution embutida), auto-update, painel de consumo, onboarding com prints	2–3 semanas

Estimativas para uma pessoa desenvolvendo com apoio de IA; servem como ordem de grandeza.

Decisões em aberto

1. Rascunho criado pelo WhatsApp **reserva estoque** no ERP ou só fica registrado?
2. Segundo fator: data de nascimento já é campo obrigatório no cadastro de cliente da Nive?
3. Evolution: começar com Docker (mais rápido) ou já investir no instalador embutido?
4. O Nive Atende será um adicional pago do plano, gratuito, ou parte de um plano superior?
5. O sistema se apresenta como assistente virtual ou fala só em nome da loja?